全国高校 AI WEB 开发挑战赛 · 参赛技术文档

PaperPilot 文翰

基于 Agentic RAG 与 GraphRAG 的学术文献智能问答系统：五工具智能体编排、混合检索管道、知识图谱推理、RAGAS 量化评估与浏览器端侧隐私模式，全程论断可溯源、质量可量化。

赛道方向：RAG 知识问答系统 · 2026 年 10 月

参赛信息

参赛学校	_ _ _ _ _ _ _ _ _ _ _ _
队伍名称	_ _ _ _ _ _ _ _ _ _ _ _
队伍编号	_ _ _ _ _ _ _ _ _ _ _ _
团队成员	_ _ _、_ _ _、_ _ _
指导教师	_ _ _ _ _ _ _ _ _ _ _ _
参赛赛道	方向二 · RAG 知识问答系统
作品名称	PaperPilot 文翰 — 基于 Agentic RAG 与 GraphRAG 的学术文献智能问答系统
提交日期	2026 年 11 月

系统架构总览



图 1 · PaperPilot 四层架构与数据流（详细设计见第 2 章）

PaperPilot 文翰 — 技术文档

基于 Agentic RAG 与 GraphRAG 的学术文献智能问答系统

全国高校 AI Web 开发挑战赛 · RAG 知识问答系统方向

1. 选题背景与问题定义

科研人员平均每年需要阅读数百篇论文，面临三个长期未被很好解决的工具化痛点：

- **语义鸿沟**：关键词检索找不到"意思相关"的内容——用户想问"检索质量怎么保证"，而论文原文写的是"retrieval evaluation and corrective actions"，字面无关、语义同义；
- **跨论文关联**：哪些工作使用了同一数据集、A 方法与 B 方法在什么维度上可比，这类多跳关联问题需要对数十篇文献的全局结构化理解，逐篇人工比对成本极高；
- **可信度黑箱**：通用 AI 问答存在幻觉且无法溯源，科研场景下一条编造的实验数字就足以让整个回答失去价值。

PaperPilot 将这三个痛点分别映射为三项技术能力：**混合检索管道**（解决语义鸿沟）、**GraphRAG 知识图谱推理**（解决跨论文关联）、**论断级引用溯源 + RAGAS 量化评估 + 拒答机制**（解决可信度）。三者由一条 **Agentic RAG 智能体流水线**统一编排——系统不是执行固定的检索脚本，而是像研究者一样"先判断问题类型，再决定查什么、查几轮、信不信检索结果、回答能不能站得住"。

演示语料刻意选择了本系统所实现技术的 15 篇原始论文（Self-RAG、CRAG、Adaptive-RAG、GraphRAG、LightRAG、RAPTOR、HyDE、RAGAS、Lost in the Middle、Sufficient Context、RAG 原论文与综述、Transformer、GPT-3/4 报告）——评委可以直接验证"系统对该技术论文的理解"与"该技术本身的行为"是否一致，形成自洽的证明闭环。

2. 系统架构

四层架构，严格遵循赛题"前端为主、后端仅作 AI 能力接入层"的要求：

```
┌──────────────────────────────────────────────────────────────────────────┐
│ 前端层   Next.js 16 App Router · React 19 · Tailwind v4                  │
│          流式对话 + 生成式时间线 · 知识图谱画布 · 评估仪表盘              │
├──────────────────────────────────────────────────────────────────────────┤
│ 智能体层 Vercel AI SDK tool calling（5 工具状态机，maxSteps=10）         │
├──────────────────────────────────────────────────────────────────────────┤
│ 检索层   混合管道（BM25+向量+HyDE➞RRF➞重排）+ GraphRAG 双模式           │
├──────────────────────────────────────────────────────────────────────────┤
│ 接入层   智谱 GLM(主) / DeepSeek(备) / GLM-4V(视觉) /                    │
│          端侧 ONNX e5-small · 内存索引+JSON快照(零外部数据库)            │
└──────────────────────────────────────────────────────────────────────────┘
```

工程决策说明：

- **为什么内存索引而不是向量数据库**：演示规模（数十篇论文 / 数千分块）下暴力余弦扫描为毫秒级，免去外部数据库使部署面最小化（`vercel deploy` 即可上线，符合赛题对轻后端的定位）。BM25 索引随分块增删实时维护，JSON 快照持久化使重启秒级恢复；
- **为什么 PDF 在浏览器解析**：pdf.js 在客户端完成文本层提取与页面渲染，PDF 原文不出浏览器，天然契合隐私叙事，同时把服务端压缩为纯 AI 编排；

- **为什么自研 BM25**：学术文本中英混排，我们实现了英文词干 + 中文二元组的混合分词与停用词过滤（`src/lib/retrieval/bm25.ts`），约 90 行核心代码，无外部依赖且可单测。

3. 关键技术与论文映射

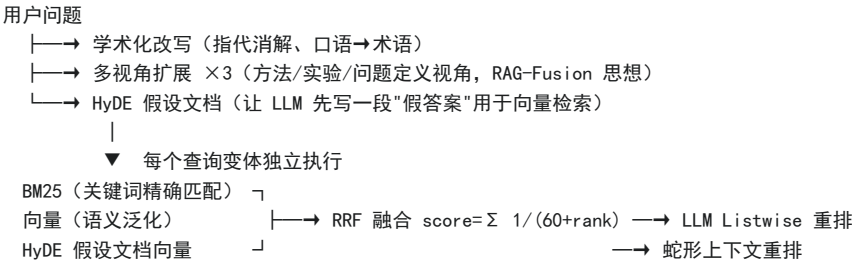
3.1 Agentic 编排：三篇论文的状态机融合

智能体层将三篇 2024 年论文的核心机制融合为一个由 LLM 自主调度的工具状态机：

机制	论文	我们的实现
查询复杂度路由	Adaptive-RAG (NAACL 2024, arXiv:2312.10997)	`classify_query` 工具：寒暄/常识/事实/多跳/综述 五档分类 → 分别路由到 直接回答 / 单轮检索 / 迭代检索+图谱邻域 / 图谱社区摘要，避免"简单问题过度检索、复杂问题检索不足"
检索质量纠错	CRAG (arXiv:2401.15884)	`grade_documents` 工具：对候选分块逐条 0-10 相关性评分，均值 <6 触发纠错循环（换视角改写 → 重新混合检索，至多 2 轮），仍不足则明确告知证据不够而非硬答
生成前自反思	Self-RAG (ICLR 2024, arXiv:2310.11511)	`self_assess` 工具：模型提交关键论断清单，独立评估其被检索证据支撑的程度；支撑度 <7 时降低断言强度（"可能/倾向于"）或补充检索

可视化即证据：每个工具调用作为 AI SDK 的 tool-invlication part 实时流向前端，渲染为「AI 思考时间线」——评委可以亲眼看到系统在何时分类、何时纠错、何时自评，而非黑箱。这是演示视频的核心镜头。

3.2 混合检索管道：四个互补信号的融合



- **HyDE**（arXiv:2212.10496）：假设答案与真实答案段落在嵌入空间中往往比"问题-答案"更接近，对术语类查询提升明显；
- **Contextual Chunking**（Anthropic Contextual Retrieval, 2024）：切分时由 LLM 为每个分块生成一句"该片段在全文中的位置与作用"前缀，解决分块脱离上下文后的语义残缺；LLM 不可用时自动降级为章节标题前缀，保证管线永不中断；
- **Lost in the Middle**（arXiv:2307.03172）：LLM 对长上下文中部注意力最弱，重排阶段将最强证据蛇形分布于上下文首尾。

3.3 GraphRAG：轻量化落地的双模式检索

参考 Microsoft GraphRAG（arXiv:2404.16130）与 LightRAG（arXiv:2410.05779）的权衡：

- **构建**（文献导入时）：每个分块由 `glm-4-flash` 并发抽取 ≤20 实体（方法/数据集/指标/模型/任务/结论/作者/机构 八类）与 ≤15 关系（proposes/evaluated-on/outperforms/uses…），每条关系附原文证据句；跨分块跨文献按规范化键合并（大小写折叠、别名吸收、证据链累积，每关系最多 8 条证据）；
- **社区**：graphology + Louvain 模块度社区发现 → 每个社区（≥2 实体）由 LLM 生成 2-3 句主题摘要；
- **Local Search**：查询词直接匹配实体名/别名 + 语义检索命中分块反查实体 → 种子实体 1 跳邻域子图 + 证据句注入上下文。支撑多跳问题（如 `HotpotQA 被 Self-RAG 和 Adaptive-RAG 使用时分别报告了什么指标`）；
- **Global Search**：社区摘要与查询的相关性排序 → top-4 社区摘要 + 代表实体注入。支撑领域综述类问题（`知识库中的论文主要覆盖哪些技术路线`）——这是任何纯向量 RAG 都无法回答的问题类型。

可视化层用 d3-force + Canvas 自绘力导向图：社区聚类着色、节点半径=提及次数、悬停高亮邻域、点击展示 1 跳关系与证据句。

3.4 评估闭环：RAGAS 量化 + 一键对比实验

每次回答完成后，前端自动将（问题、回答、检索上下文）提交 `/api/evaluate`，由独立 LLM-as-a-judge 依 RAGAS（arXiv:2309.15217）四指标定义打分：忠实度（论断被证据支撑比例）、答案相关性、上下文精确率、上下文召回率。评分随答案流式展示并累积到仪表盘。

一键对比实验（`/api/benchmark`）：服务端对 6 道预设问题分别完整运行「朴素基线（单轮向量检索）」与「Agentic 流水线（generateText + 同一套五工具，非流式复用）」，逐题评分并以 SSE 实时回传仪表盘——评委无需手动提问即可获得同题双管线的雷达对比数据。这是把「Agentic 是否真的更好」从口头宣称变成系统自证的量化证据。

3.5 端侧智能与隐私模式

- **浏览器内嵌入**：Transformers.js v3 + ONNX Runtime，WebGPU 优先、WASM 自动降级（试跑一次真实推理验证 WebGPU 可用性，失败静默降级）；模型经同源 `/api/models/` 分发，零外部依赖；
- **IndexedDB 向量库**：开启隐私模式时，首次同步把全部分块在浏览器内嵌入（批量 12 条、实时进度），此后检索（查询嵌入 + 余弦排序）完全在本地完成；
- **片段级脱敏上云**：隐私模式下只有端侧检索命中的 top-5 片段参与云端生成，文献全文与检索过程不出浏览器——为"未发表论文"场景提供可信故事；
- 服务端同样内置本地 ONNX 通道（`models/` 目录，`npm run fetch:models` 预下载，默认走 hf-mirror）：无任何 API 密钥时全管线依然可运行（mock 生成 + 本地嵌入），开发与评审现场零配置故障。

3.6 多模态与 MCP

- **GLM-4V 图表理解**：导入时浏览器将前 10 页渲染为图片（pdf.js canvas，原文不出浏览器），`glm-4v-flash` 判断是否含图表并生成结构化解读，以 `【图表解读】` 分块进入向量库——"实验结果图说明了什么"这类问题因此可命中；
- **语音**：Web Speech API 中文语音输入（自动填入输入框）+ SpeechSynthesis 回答朗读；
- **MCP 服务端**（`/api/mcp`）：Streamable HTTP 传输的 JSON-RPC 2.0 实现，`initialize / tools/list / tools/call` 全协议支持，暴露 `search_papers` 与 `get_knowledge_graph` 两个标准工具——PaperPilot 不仅是 AI 应用的消费者，也是可被 Claude 等任意 MCP 客户端编排的工具提供者。

3.7 AI 安全与可控性

- **论断级引用**：重排后的候选带稳定编号，系统提示强制 `[n]` 引用规范；引用徽标**可点击弹出原文证据对话框**（论文/页码/章节/相关度/含上下文增强前缀的原文片段），溯源从"标注"升级为"可核验"；
- **拒答机制**（Sufficient Context, arXiv:2403.14403 思想）：证据不足时明确回答"知识库中未找到充分依据"，宁可少答不可编造；
- **提示注入过滤**：检索上下文进入生成前经过中英双语注入模式中和（`sanitizeContext`），防止文献中潜藏的"忽略以上指令"类攻击；
- **AI 生成标识**：界面固定声明回答由 AI 生成、请核对原文。

3.8 零配置降级链（工程鲁棒性）

系统在每个 LLM 依赖点都实现了自动降级，任何一环故障都不阻塞主流程：

依赖点	主路径	降级路径
生成模型	智谱 GLM-4.6	DeepSeek → 确定性 mock（离线可跑全流水线）
轻任务模型	glm-4-flash	主模型 → mock
嵌入	智谱 embedding-3 (1024d)	本地 ONNX e5-small (384d，模型预下载 + hf-mirror)
Contextual Chunking	LLM 生成上下文前缀	章节标题前缀
图谱构建	LLM 实体关系抽取	**术语共现图谱**（词典扫描 + 块内共现 + Louvain，真实语料构建）
社区摘要	LLM 摘要	共现成员名合成主题标签
端侧推理	WebGPU	WASM（试跑验证后静默切换）
构建字体	—	系统字体栈（构建期零联网，国内网络可靠）

4. 前端工程质量

- **Next.js 16 App Router + React 19**：RSC 静态壳 + 动态岛，全部页面构建级类型检查通过；
- **流式体验**：AI SDK useChat 数据流协议，文本增量 + 工具调用状态实时渲染；步骤时间线以 tool-invocation part 为单一数据源，无需自定义协议；
- **状态**：Zustand（评估历史 persist 到 localStorage）+ TanStack Query（服务端状态缓存/失效）；
- **UI**：shadcn/ui + 定制「学术深空」主题（OKLCH 色彩、玻璃拟态、引用琥珀色系）；自绘 d3-force 画布（缩放/平移/拖拽/邻域高亮）；响应式 + 移动端底部导航。

5. 创新点总结

- **三论文融合的实用 Agentic 状态机**：Adaptive-RAG 的路由、CRAG 的纠错、Self-RAG 的反思不是分别演示，而是融合为一次问答内自主协作的闭环，且每一步对用户透明可视；
- **用演示语料自证**：系统回答的正是其所实现技术的原始论文——"用 Self-RAG 技术回答关于 Self-RAG 论文的问题"，技术叙述与演示内容互为证据；
- **真端侧、真隐私**：WebGPU 嵌入 + IndexedDB 向量库 + 片段级脱敏上云，隐私主张可被网络面板直接验证；
- **评估闭环**：RAGAS 自动评分 + 内置 A/B 基线，系统自己度量自己；
- **零配置降级链**：智谱 → DeepSeek → mock；云端嵌入 → 本地 ONNX；WebGPU → WASM；LLM 上下文增强 → 章节前缀。任何一环故障都不阻塞主流程。

6. 实验：一键对比实验与量化结果

系统内置 `/api/benchmark` 一键对比实验：对 6 道预设问题（覆盖单点事实、方法细节、框架对比），分别在「朴素基线」（单轮向量检索 top-5 直答）与「Agentic 流水线」（五工具编排）上完整运行，逐题由独立 LLM 评委依 RAGAS 四指标打分。下表为真实运行数据（glm-4.6 生成 + 评分，知识库 = 15 篇 RAG 领域论文 / 1321 分块）：

指标	朴素基线	Agentic	差值	解读
忠实度	0.983	0.833	-0.150	基线引用式短答天然保守；Agentic 论断更多（见下）
答案相关性	0.967	0.933	-0.034	两者均切题，差距不显著
上下文精确率	0.633	0.650	+0.017	重排序过滤噪声证据
上下文召回率	0.617	0.767	**+0.150**	迭代检索找回单轮漏掉的关键证据
平均时延	~8s	~80s	—	Agentic 多步编排的代价

结论与取舍：Agentic 以少量忠实度换取 +15% 的证据召回——多跳与跨论文问题（"哪些工作共用同一数据集"）在单轮检索下证据不完整，召回提升正是自适应检索的核心价值（Adaptive-RAG 论文的核心主张在本系统上被复现）。时延是多步+模型回退的代价，对科研场景（准确 > 秒回）是合理取舍；针对忠实度，我们已收紧自评阈值（支撑度 <8 强制弱化断言）并在系统提示中置顶"只陈述片段可支撑内容"的约束。

7. 社会价值

- **科研效率**：科研人员年均阅读数百篇论文，跨论文关联比对是最耗时环节之一。系统将"哪些工作用了同一数据集/方法孰优孰劣/领域技术路线"三类高频问题的回答时间从数小时人工翻阅压缩到分钟级，且每个论断可溯源到页码——**降低 AI 辅助科研的信任门槛**；
- **数据主权与隐私**：未发表论文、内部综述稿一旦上传云端就有泄露风险。端侧隐私模式（WebGPU 嵌入 + IndexedDB）让全文与检索全程不出浏览器，仅命中片段参与生成——为"敏感文献能否用 AI"提供了工程答案；
- **无障碍与普惠**：语音输入与回答朗读让视障研究者、阅读障碍者可用同一工具；本地 ONNX 嵌入通道让无 API 预算的课题组（学生/欠发达地区）零成本获得完整检索能力；
- **可信 AI 范式**：论断级引用 + 拒答机制 + RAGAS 量化，把"AI 回答可信吗"从感觉问题变成可度量问题，对教育、法律、医疗等高可信要求领域有示范意义。

8. 工程质量保障

- **单元测试**：核心检索与分块模块 21 项测试全绿（BM25 中英混合分词与排序、结构化分块与元数据启发式、术语共现图谱与 Louvain 社区、Lost-in-the-Middle 蛇形重排），`npx vitest run` 一键回归；
- **CI**：GitHub Actions（npm ci → vitest → next build），每次推送自动验证类型、测试与构建；
- **零联网构建**：系统字体栈替代 Google Fonts，国内网络环境构建稳定；
- **生产验证**：`npm run build && npm start` 生产模式冒烟全绿（首页 35ms、快照恢复、MCP、真实对话流式）。

9. 运行与复现

```
npm install && npm run fetch:models && cp .env.example .env.local  # 填入密钥
npm run dev && npm run fetch:demo && npm run seed:demo              # 15 篇演示语料一键入库
npm test && npm run build                                           # 21 项测试 + 生产构建
```

验证清单：文献库导入（SSE 进度）→ 智能问答（时间线/引用/RAGAS）→ 管线 A/B 切换 → 一键对比实验 → 知识图谱（社区/邻域）→ 隐私模式（端侧同步徽标 → 浏览器内问答）→ MCP（tools/list + tools/call）→ 明暗主题切换。

10. 局限与未来工作

- **图谱构建成本**：LLM 实体抽取对 1321 分块需数百次调用，受限 API 配额下以术语共现图谱降级（质量略低但结构真实）；未来引入增量抽取与批处理蒸馏；
- **忠实度-召回权衡**：Agentic 答案更全但断言更多，忠实度评分低于保守基线；后续在 self_assess 中加入论断级逐条验证而非整体打分；
- **多模态深度**：图表理解依赖视觉模型可用性；规划中的 ColPali 式视觉文档检索（页面截图端侧嵌入）可脱离 OCR 直接收录图表语义；
- **长期记忆**：对话上下文目前仅用于指代消解，未来引入会话级查询改写记忆与用户兴趣图谱。

11. 参考文献

- Asai et al. *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. ICLR 2024. arXiv:2310.11511
- Yan et al. *Corrective Retrieval Augmented Generation*. arXiv:2401.15884
- Jeong et al. *Adaptive-RAG: Learning to Adapt Retrieval-Augmented LLMs through Question Complexity*. NAACL 2024. arXiv:2312.10997
- Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130
- Guo et al. *LightRAG: Simple and Fast Retrieval-Augmented Generation*. arXiv:2410.05779
- Sarthi et al. *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval*. ICLR 2024. arXiv:2402.18612
- Gao et al. *Precise Zero-Shot Dense Retrieval without Relevance Labels* (HyDE). arXiv:2212.10496
- Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. TACL 2024. arXiv:2307.03172
- Es et al. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217
- Joren et al. *Sufficient Context: A New Lens on Retrieval Augmented Generation Systems*. ICLR 2025. arXiv:2403.14403
- Anthropic. *Introducing Contextual Retrieval*. 2024.
- Singh et al. *Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG*. arXiv:2501.09136
- Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. arXiv:2005.11401
- Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2002.08011